Implementing Replicated Logs with Paxos

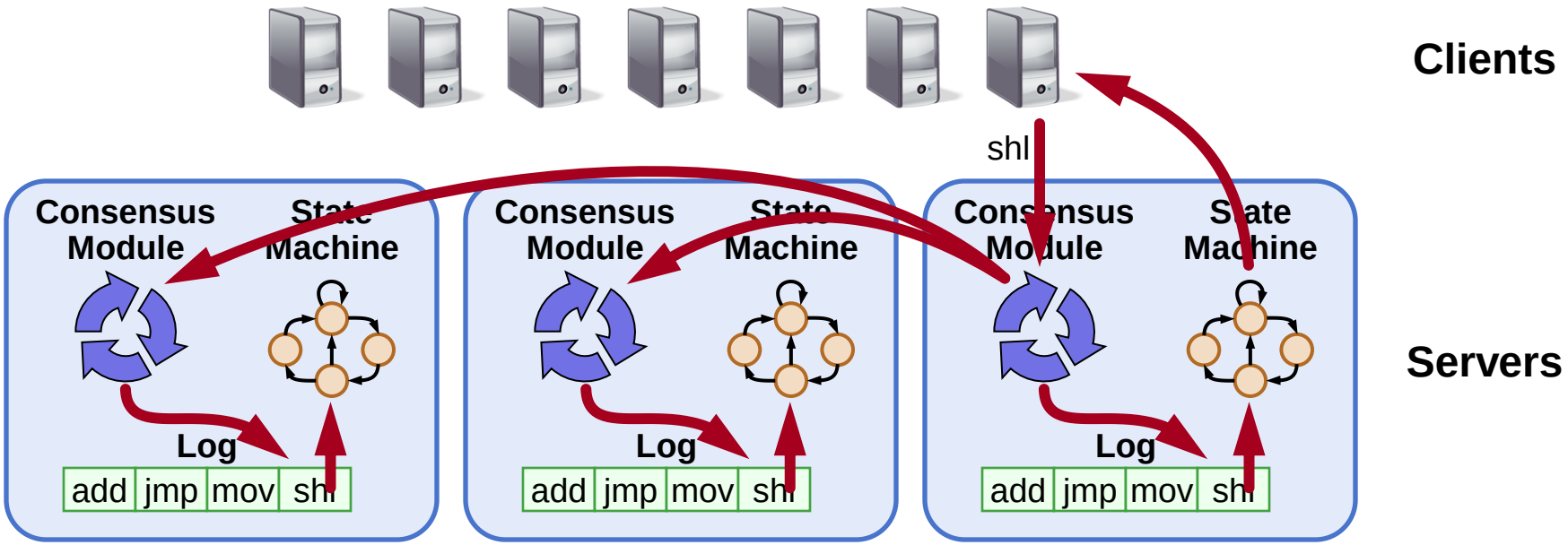
John Ousterhout and Diego Ongaro

Stanford University



Дополнения / изменения: Косяков Михаил Сергеевич, к.т.н., доцент кафедры ВТ

Goal: Replicated Log



- **Replicated log => replicated state machine**
  - All servers execute same commands in same order
- **Consensus module ensures proper log replication**
- **System makes progress as long as any majority of servers are up**
- **Failure model: fail-stop (not Byzantine), delayed/lost messages**

Replicated log

- Total order для commands эквивалентно определению total order для реквестов входа в КС (mutual exclusion)
  - Команды доставляются и исполняются в порядке входов в КС
- Централизованный алгоритм для total order:
  - Между координатором и остальными узлами каналы FIFO
  - Процесс отправляет своё сообщение / команду координатору (соответствует запросу на вход в КС)
  - Координатор просто рассылает сообщение / команду получателям (соответствует разрешению на вход в КС)
- Централизованный алгоритм для causal total order
  - К запросу добавляем векторное время отправителя
  - Координатор рассылает сообщение, когда оно становится eligible

ITIVITI
LET IT TRADE
УНИВЕРСИТЕТ ИТМО

Replicated log

- Алгоритм Лэмпорта для causal total order:
  - Броадкастим timestamped команду (запрос на вход в КС)
  - При получении кладем timestamped команду в очередь и возвращаем timestamped ACK
  - Инициатор исполняет команду и удаляет ее из очереди по правилам алгоритма Лэмпорта (вход в КС)
  - Инициатор рассылает всем уведомление, что можно исполнять команду (выход из КС)
- Алгоритм Рикарта-Агравала для causal total order:
  - Инициатор исполняет команду, когда соберет все разрешения, откладывая свои разрешения командам с бОльшим timestamp
  - Инициатор рассылает отложенные разрешения, а также уведомление всем, что можно исполнять команду

ITIVITI
LET IT TRADE
УНИВЕРСИТЕТ ИТМО

Replicated log

ITIVITI

- Алгоритмы взаимного исключения Лэмпорта и Рикарта-Агравала не fault-tolerant
- В алгоритме Лэмпорта мы знаем, что можно исполнять команду, когда получили все ACK с timestamp больше нашего (в алгоритме P-A просто *все* ACK)
  - если мы не получили от кого-то ACK, то не можем исполнять нашу команду: вдруг он не умер, а от него едет команда с timestamp меньшим нашего (и эта команда уже подтверждена остальными и исполнена)
- В алгоритме P-A конфликты доступа к КС разрешались непосредственно самими инициаторами входа в КС
  - в Paxos конфликты разрешаются (одним или более) acceptors из пересечения majorities (quorum-based алгоритм)

LET IT TRADE

УНИВЕРСИТЕТ ИТМО

Quorum-based алгоритмы

ITIVITI

- Почему quorum-based алгоритмы не требует разрешения от всех участников и при этом работают корректно?
  - Инициатор знает что можно исполнять команду, когда соберет разрешения с произвольного majority (кворума в общем случае)
  - Для двух конкурирующих команд будет существовать как минимум один процесс из пересечения majorities, который будет знать об обеих командах
  - Для третьей конкурирующей команды majority будет включать либо процесс, который видел первые две команды, либо два процесса, один из которых видел первую команду, другой – вторую
- Т.о. любое majority коллегиально будет иметь информацию о всех реквестах, контактировавших с любыми другими majority
  - Потому что пересечение любых двух majorities непусто
  - Это позволяет любому majority сделать выбор в пользу одного реквеста из всех увиденных

LET IT TRADE

УНИВЕРСИТЕТ ИТМО

The Paxos Approach

The Part-Time Parliament

LESLIE LAMPORT
Digital Equipment Corporation

Recent archaeological discoveries on the island of Paxos reveal that the parliament functioned despite the peripatetic propensity of its part-time legislators. The legislators maintained consistent copies of the parliamentary record, despite their frequent forays from the chamber and the forgetfulness of their messengers. The Paxon parliament's protocol provides a new way of implementing the state-machine approach to the design of distributed systems.

Categories and Subject Descriptors: C2.4 [**Computer-Communications Networks**]: Distributed Systems—*Network operating systems*; D4.5 [**Operating Systems**]: Reliability—*Fault-tolerance*; J.1 [**Administrative Data Processing**]: Government

General Terms: Design, Reliability

Additional Key Words and Phrases: State machines, three-phase commit, voting

This submission was recently discovered behind a filing cabinet in the *TOCS* editorial office. Despite its age, the editor-in-chief felt that it was worth publishing. Because the author is currently doing field work in the Greek isles and cannot be reached, I was asked to prepare it for publication.

The author appears to be an archeologist with only a passing interest in computer science. This is unfortunate; even though the obscure ancient Paxon civilization he describes is of little interest to most computer scientists, its legislative system is an excellent model for how to implement a distributed computer system in an asynchronous environment. Indeed, some of the refinements the Paxons made to their protocol appear to be unknown in the systems literature.

The author does give a brief discussion of the Paxon Parliament's relevance to dis-

LET IT TRADE

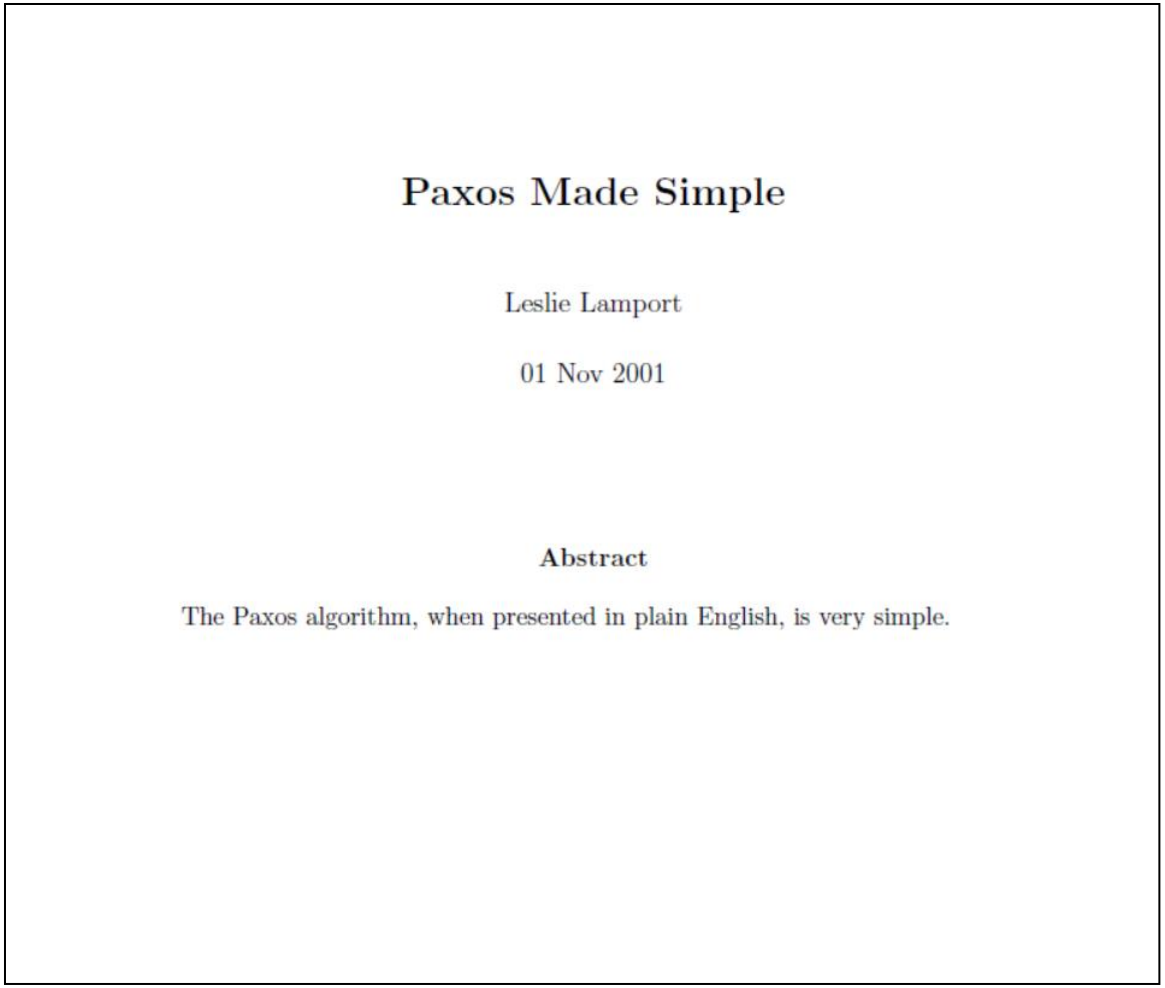
ITIVITI

УНИВЕРСИТЕТ ИТМО

The Paxos Approach

ITIVITI

LET IT TRADE



Paxos Made Simple

Leslie Lamport

01 Nov 2001

Abstract

The Paxos algorithm, when presented in plain English, is very simple.

УНИВЕРСИТЕТ ИТМО

The Paxos Approach

ITIVITI

- Basic Paxos («single decree»):
  - Есть совокупность процессов, которые могут предлагать (**propose**) значения
  - Алгоритм консенсуса должен обеспечивать выбор одного-единственного значения (**chosen**) среди предлагаемых (**proposed**)
  - Если значение выбрано (**chosen**), процессы должны иметь возможность узнать выбранное значение (**learn**)
- Multi-Paxos:
  - Последовательность отдельных инстансов алгоритма Basic Paxos: значение, выбранное для *i*-го инстанса, является *i*-ой командой state-машины в replicated log

LET IT TRADE

УНИВЕРСИТЕТ ИТМО

Basic Paxos: Требования

ITIVITI

- Безопасность (safety):
  - Только одно значение может стать выбранным (agreement)
  - Только какое-либо из предложенных значений может стать выбранным (validity)
  - Процессы не должны думать, что значение выбрано до тех пор пока оно на самом деле не станет выбранным
- Живучесть (liveness):
  - Какое-то из предложенных значений рано или поздно станет выбранным и, если выбор состоялся, то процессы рано или поздно узнают (learn) о выбранном значении

LET IT TRADE

УНИВЕРСИТЕТ ИТМО

Basic Paxos: Роли

ITIVITI

- Proposers (лидеры, координаторы):
  - Активные: принимают команды от клиентов и делают предложения, двигают алгоритм вперед, чтобы acceptor-ы пришли к консенсусу
- Acceptors:
  - Пассивные: делают то, что им сказали proposer-ы, отвечают на их сообщения, голосуют (**accept**) за то или иное предложение
  - Хранят состояние алгоритма и выбранное значение
- Learners:
  - Хотят узнать выбранное значение
- Multi-Paxos:
  - Каждый сервер играет все три роли: proposer, acceptor и learner для каждого инстанса алгоритма

LET IT TRADE

УНИВЕРСИТЕТ ИТМО

Basic Paxos: Один Acceptor

ITIVITI

- Чтобы не было конфликтов выбора, лучше бы, чтобы proposer был один, но если есть конфликтующие предложения – не беда, консенсус для этого и нужен, чтобы выбрать одно предложение
- Самый простой способ выбрать значение – один Acceptor
  - Proposer-ы отправляют предложения этому Acceptor-у
  - Acceptor выбирает первое полученное предложение
  - Not fault-tolerant: сбой Acceptor-а блокирует протокол (а если Acceptor выходит из строя сразу после того, как выбрал значение, то мы и значения этого не узнаем)
- Вывод: нужно больше Acceptor-ов !

LET IT TRADE

УНИВЕРСИТЕТ ИТМО

Basic Paxos: Множество Acceptor-ов

ITIVITI

LET IT TRADE

- В Paxos значение будет выбранным (chosen) когда произвольное большинство (majority) acceptor-ов проголосуют за него (accept):
  - Любые два majority имеют как минимум один acceptor в пересечении
  - Можно гарантировать, что только одно значение наберет большинство, если acceptor голосует (accept) **максимум за одно** значение
- В Paxos *f* acceptor-ов из общего числа *2f* +1 может отказать, и алгоритм продолжит корректно работать
  - Набор из *2f* + 1 acceptor-ов фиксирован (но можно изменить с помощью самого Paxos-а!)
  - Proposer-ы могут приходить и уходить в любой момент

УНИВЕРСИТЕТ ИТМО

Basic Paxos: Множество Acceptor-ов

ITIVITI

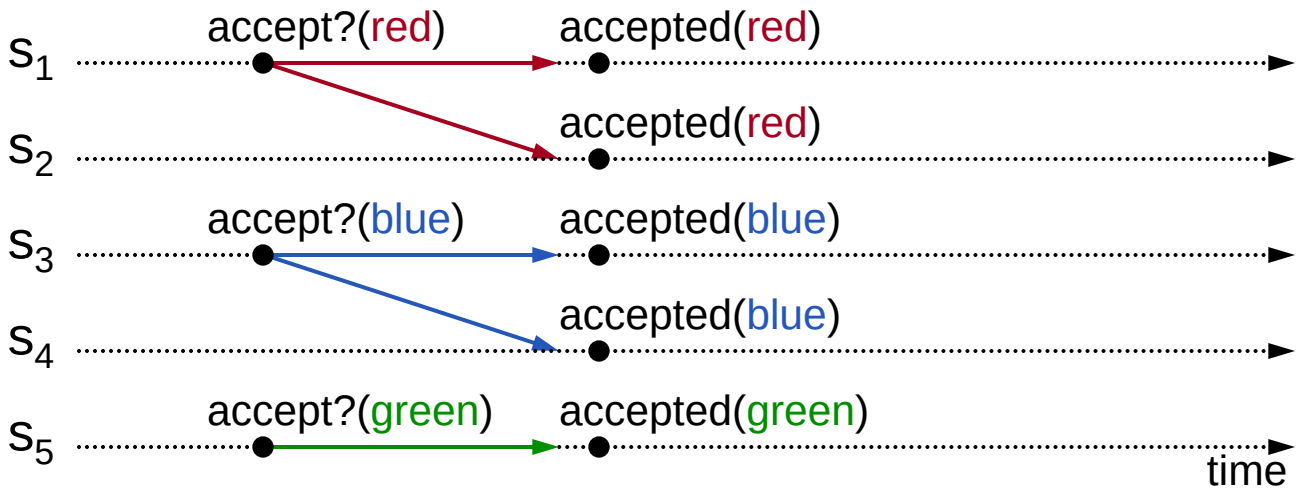
LET IT TRADE

- Но может ли каждый acceptor голосовать максимум за одно значение?
- В отсутствие сбоев и потери сообщений, значение должно стать выбранным даже если оно является единственно предложенным. Отсюда вытекает **Правило 1**…
- **П1:** Acceptor должен голосовать за первое полученное предложение
- Т.о. acceptor голосует за первое полученное предложение и только его?

УНИВЕРСИТЕТ ИТМО

Problem: Split Votes

- **Acceptor accept-ит только первое полученное предложение?**
- **Выбор может не состояться (ни одно значение не accepted большинством):**



- **Выбор может не состояться в т.ч. из-за потери сообщений и из-за сбоев acceptor-ов (даже если всего два предложения)**

Basic Paxos: Сбой acceptor-ов

ITIVITI

- Интересная ситуация: половина acceptor-ов (s_1 и s_2) проголосовала за **red**, половина (s_4 и s_5) за **blue**
- s_3 не дышит, не успев сообщить результаты своего accept-а, т.е. алгоритм не завершился с т.з. proposer-ов **red** и **blue**
- Непонятно как продолжать алгоритм дальше (алгоритм оказывается блокированным на s_3):
  - Если s_3 проголосовал за **red**, то алгоритм должен завершиться выбором **red** (когда s_3 восстановится, он сообщит о своем голосе за **red**)
  - Наоборот, если s_3 проголосовал за **blue**, то алгоритм должен завершиться выбором **blue**
  - Если s_3 еще не успел проголосовать или проголосовал за **green**, то выбранного значения нет, и можно выбрать что угодно, даже **brown**

LET IT TRADE

УНИВЕРСИТЕТ ИТМО

Basic Paxos: Сбой acceptor-ов

ITIVITI

- Интересная ситуация номер «два»: acceptor s_3 рапортует, что проголосовал за **red**, s_4 и s_5 говорят, что не голосовали (не получали предложений)
- Вопрос: **red** выбрано или нет? Можем ли выбрать **blue**?
- Paxos решает подобные проблемы с помощью последовательности **раундов** (rounds) для выборов
  - Это позволяет алгоритму двигаться вперед, несмотря на отказы
- Раунд ***N*** представлен единственным лидером, который пытается получить большинство голосов для значения V_N
  - Если раунд терпит неудачу, то следующий раунд может помочь
  - Если раунд успешен, т.е. собирается большинство, то значение V_N становится выбранным
- Как определяется раунд?

LET IT TRADE

УНИВЕРСИТЕТ ИТМО

Basic Paxos: Разные раунды / предложения



- Чтобы в раунде было не больше одного лидера, сделаем идентификатор лидера частью номера раунда:

  N = <seq_number, Proposer_ID>

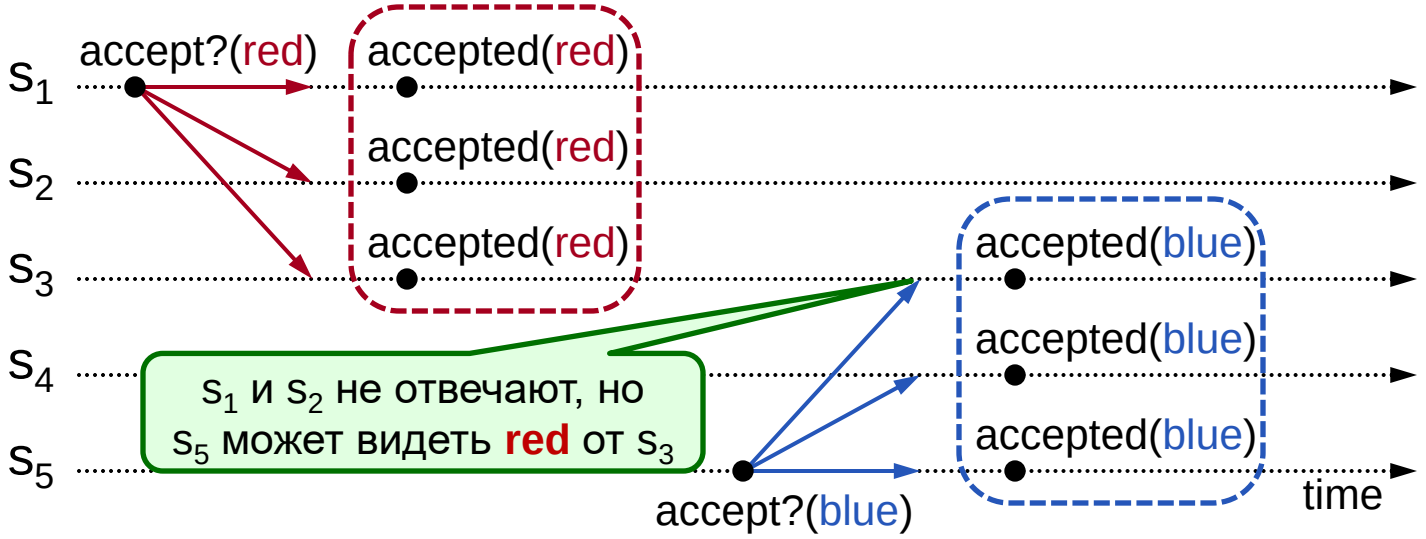
- Proposer хранит maxRound – наибольший seq_number, что он видел через поступающие ему сообщения
  - Вместо seq_number можно использовать значение физических часов (если они никогда не пойдут назад)
- Для создания нового предложения proposer инкрементирует maxRound и присоединяет к нему свой Proposer_ID
  - Вместо Proposer_ID можно использовать network address
- Чтобы в каждом раунде ***N*** было не больше одного значения V_N, proposer не должен переиспользовать maxRound:
  - Либо персистить maxRound, либо опросить произвольное большинство acceptor-ов (будет ясно позже)

LET IT TRADE

Problem: Conflicting Choices

- **Последовательность раундов делает алгоритм устойчивым к отказам (как в «интересных примерах»), но вызывает другую проблему**
- **Выборы, сделанные в разных раундах, должны совпадать**

Red Chosen: s_1 думает, что он собрал большинство голосов

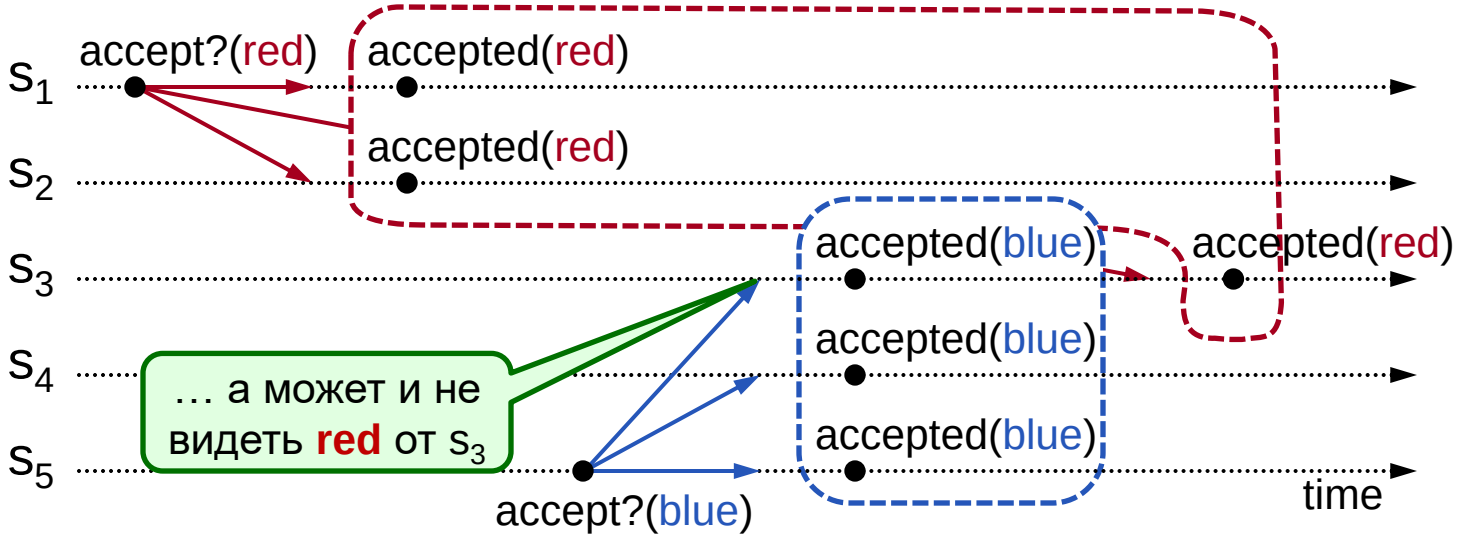


Blue Chosen: s_5 думает, что он собрал большинство голосов

Problem: Conflicting Choices

- **Последовательность раундов делает алгоритм устойчивым к отказам (как в «интересных примерах»), но вызывает другую проблему**
- **Выборы, сделанные в разных раундах, должны совпадать**

Red Chosen: s_1 думает, что он собрал большинство голосов



Blue Chosen: s_5 думает, что он собрал большинство голосов

Basic Paxos: Разные раунды / предложения

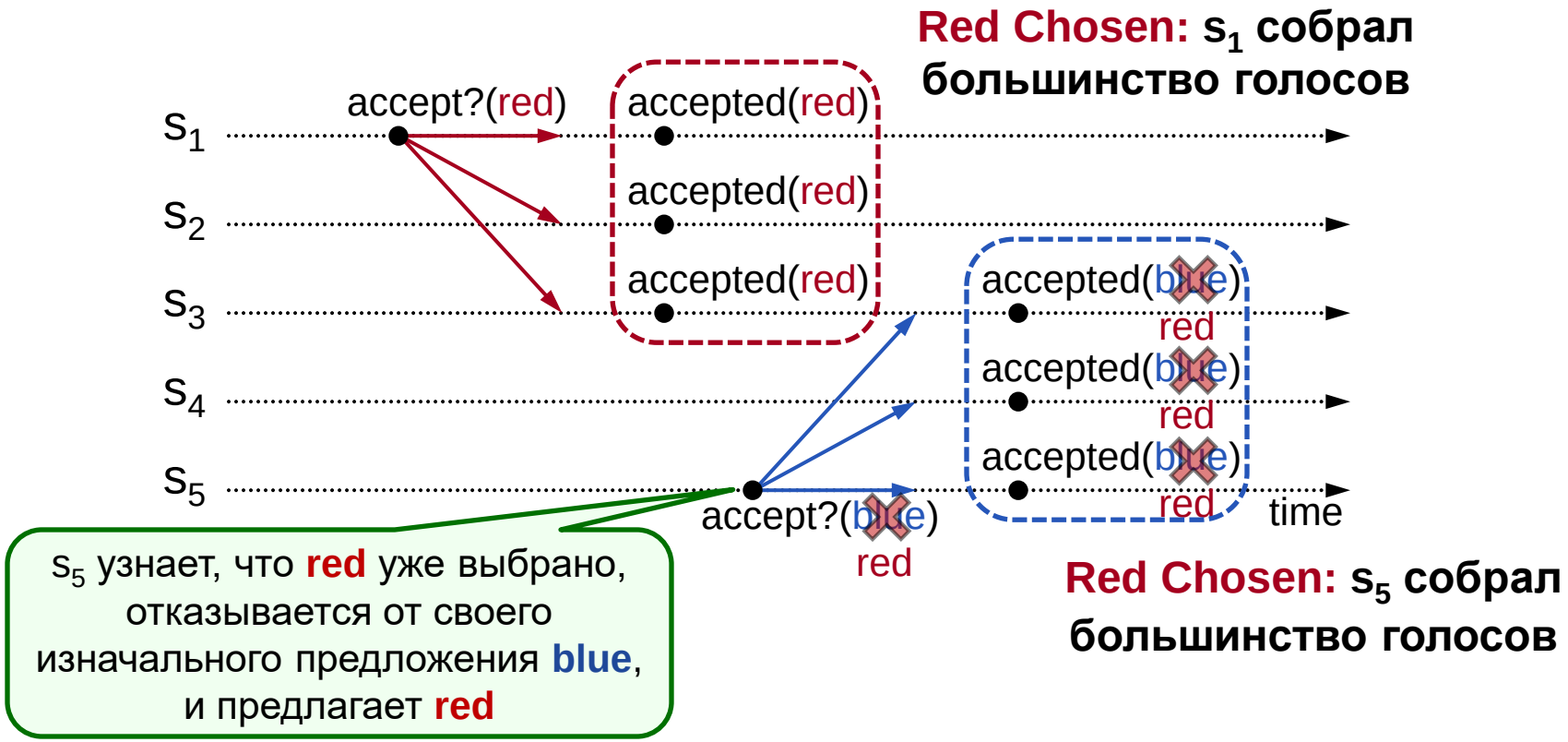


LET IT TRADE

- Основное требование безопасности: любые два успешные раунда должны выбрать одно и то же значение
- **П2**: Если предложение ***M*** со значением V_M выбрано, то **любое выбранное** предложение с бОльшим номером $N > M$ также должно иметь значение V_M
- **П2б:** Если предложение ***M*** со значением V_M выбрано, то **любое сделанное** предложение с бОльшим номером $N > M$ также должно иметь значение V_M
- **Вывод:** протокол для proposer-ов должен быть двухфазным:
  - Фаза 1, чтобы попытаться понять какое значение выбрано в предыдущих раундах
  - Фаза 2, чтобы предложить значение, (возможно) выбранное в предыдущих раундах, если такое есть, или своё собственное, если такого нет

Problem: Conflicting Choices

- **s_5 должен предложить red т.к. red уже выбрано**



- **Требование безопасности:** два разных предложения становятся выбранными, но они оба имеют одно и то же значение **red**

Basic Paxos: Разные раунды / предложения



- Но как показано в «интересных примерах» выше, понять выбрано ли значение в каком-либо раунде или нет - непросто
- Ключевая особенность Paxos в том, что предложению ***N*** не надо знать выбрано ли какое-либо предложение предыдущего раунда ***M*** или нет
- Достаточно либо знать, что раунд ***M*** гарантировано неуспешный, либо переиспользовать значение V_M этого раунда, которое окажется выбранным, если раунд ***M*** окажется успешным
  - Гарантированно неуспешный раунд ***M***: большинство не проголосовало и не проголосует за V_M
  - Гарантировано успешный раунд ***M***: большинство проголосовало за V_M, т.е. значение V_M выбрано
  - Сомнительный раунд ***M***: видим V_M, но не на большинстве

LET IT TRADE

Basic Paxos:
Разные раунды / предложения



LET IT TRADE

	Предлагаемое значение	s1	s2	s3
Раунд 1	**red**	**red**	no	no
Раунд 2	**blue**	**blue**	no	no
Раунд 3	**green**	no	no	**green**
Раунд 4	**?**			

{s1, s2, s3} – any
{s1, s2} – blue
{s1, s3} – green
{s2, s3} – green

	Предлагаемое значение	s1	s2	s3
Раунд 1	**red**	**red**	no	no
Раунд 2	**blue**	**blue**	no	**blue**
Раунд 3	**blue**	no	no	**blue**
Раунд 4	**?**			

{s1, s2, s3} – blue
{s1, s2} – blue
{s1, s3} – blue
{s2, s3} – blue



УНИВЕРСИТЕТ ИТМО

Basic Paxos: Алгоритм proposer-а

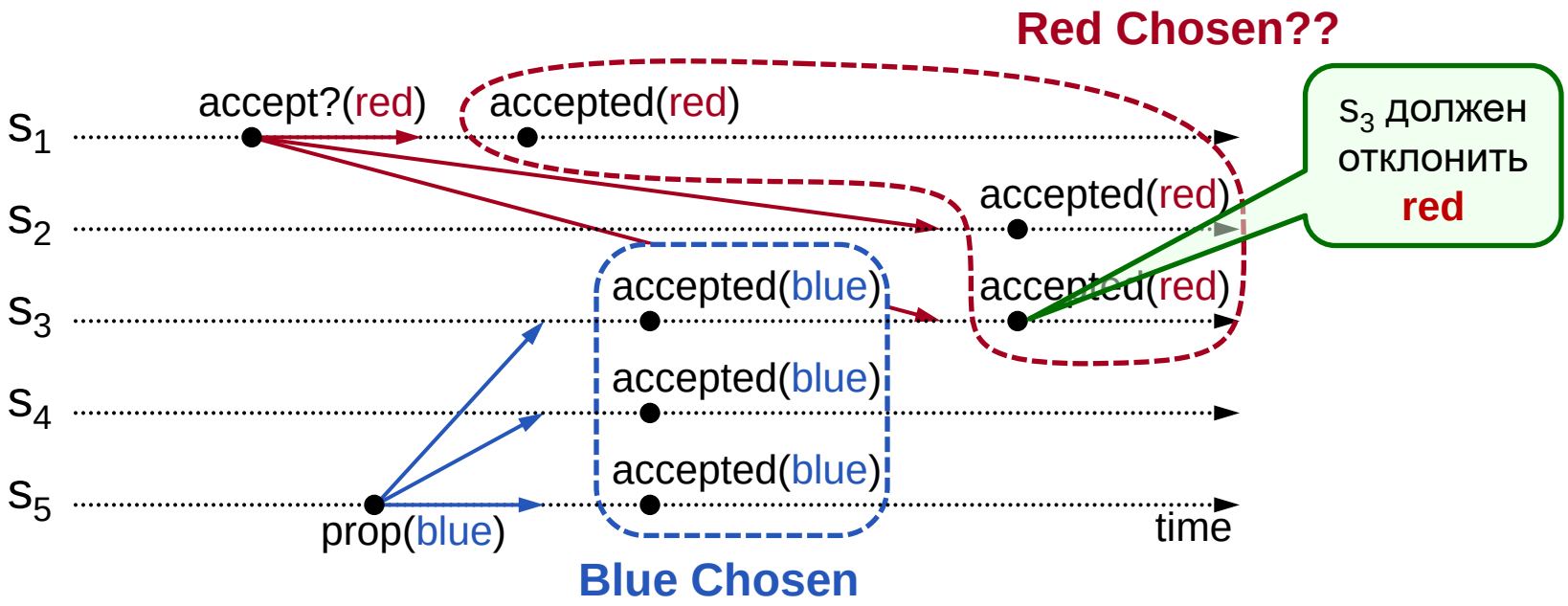


- Если предложение с номером $M < N$ и значением V_M выбрано, то существует большинство acceptor-ов C, каждый из которых проголосовал за (M, V_M)
- По индукционному предположению все предложения с номерами $M..(N - 1)$ также имеют значение V_M
  - Тогда каждый acceptor из C проголосовал за предложение с номером из диапазона $M..(N - 1)$ и значением V_M
- Если на Фазе 1 мы получим ответы от произвольного большинства S, то предложения из пересечения S и C, будут иметь номера не меньше M и значение V_M
- Поэтому если мы возьмем предложение с **наибольшим номером** среди всех accepted предложений в S мы узнаем ранее выбранное значение V_M
- Победа?

LET IT TRADE

УНИВЕРСИТЕТ ИТМО

Conflicting Choices, cont'd



- Предложение с бОльшим номером не обязательно становится выбранным **после** предложения с меньшим номером
- s_5 на Фазе 1 не видит никаких accepted предложений и поэтому не может предложить **red** и предлагает **blue**
- По **П2б** конкурирующее предложение **red** с меньшим номером не должно стать выбранным, т.е. должно быть заблокировано

Basic Paxos: Алгоритм proposer-а

ITIVITI

LET IT TRADE

- Что делать с предложением ***M*** < ***N***, которое мы не увидим на Фазе 1 и посчитаем гарантированно неуспешным, но которое может появиться на acceptor-ах из ***S*** «в будущем» и стать выбранным (возможно с другим значением V_M)?
- Не позволим таким предложениям стать выбранными: на Фазе 1 предложения ***N*** попросим acceptor-ов из ***S*** больше не голосовать за предложения с номерами < ***N***
  - Сомнительные раунды могут уже быть успешными или могут еще стать успешными «в будущем» (или не быть и не стать)
- Т.о. на Фазе 1, обращаясь к произвольному большинству, proposer пытается узнать значение (возможно) успешного раунда и зафиксировать гарантировано неуспешные раунды
- На Фазе 2, обращаясь к возможно другому большинству, proposer просит acceptor-ов проголосовать за своё или обнаруженное на Фазе 1 значение

УНИВЕРСИТЕТ ИТМО

Basic Paxos: Алгоритм acceptor-а

ITIVITI

- Acceptor получает два типа запросов от proposer: Prepare(***N***) на Фазе 1 и Accept(***N***, V_N) на Фазе 2
- Acceptor может игнорировать любые запросы без нарушения safety (но хотелось бы еще liveness)
  - На Prepare можно отвечать всегда, голосовать и отвечать на Accept можно только если Acceptor не обещал не делать этого
- **П1а:** Acceptor должен голосовать за Accept(***N***, V_N) тогда и только тогда, когда он не отвечал на Prepare(***K***) с $K > N$
  - **П1а** подразумевает **П1**
- … и не голосовал за Accept(***K***, V_k)
  - Проголосовав за Accept(***K***, V_k) также дается обещание не голосовать за предыдущие предложения Accept(***N***, V_N)

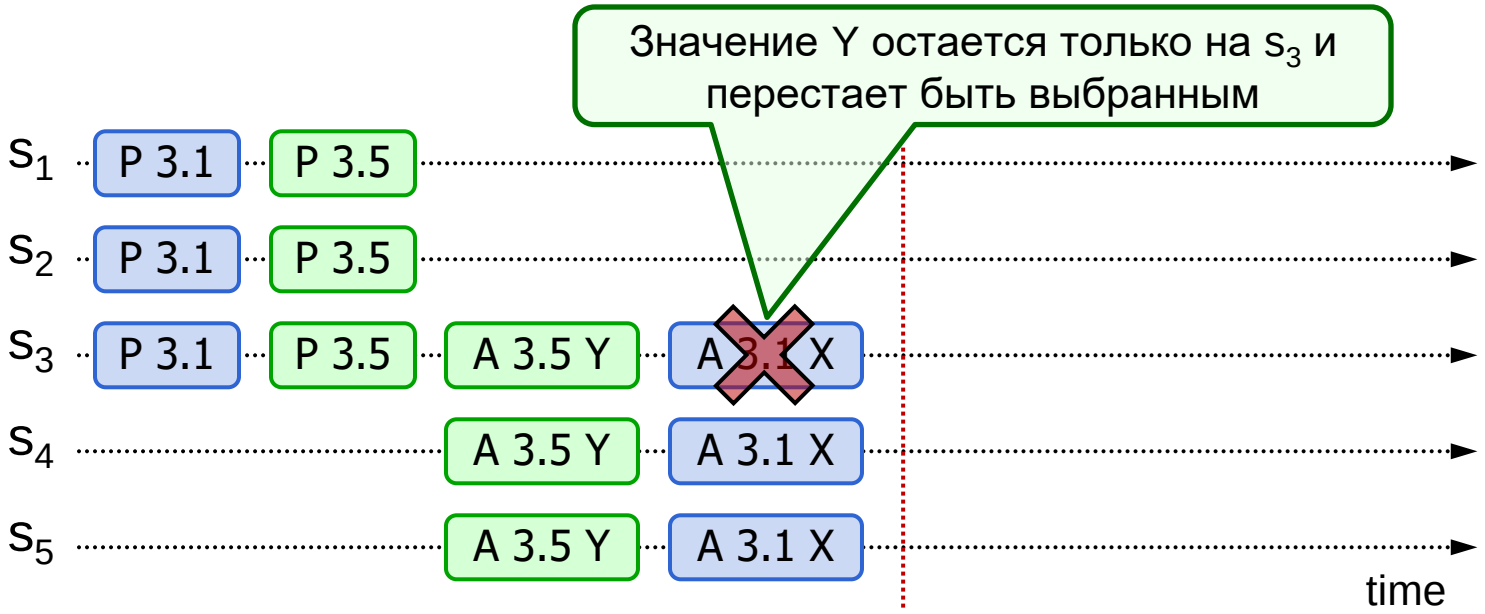
LET IT TRADE

УНИВЕРСИТЕТ ИТМО

Basic Paxos
Алгоритм acceptor-a

ITIVITI

- Если голосование за Accept(***K***, V_k) не дает обещания не голосовать за предыдущее предложение Accept(***N***, V_N), где ***N*** < ***K***, то выбранное значение V_K может перестать быть выбранным и может пропасть

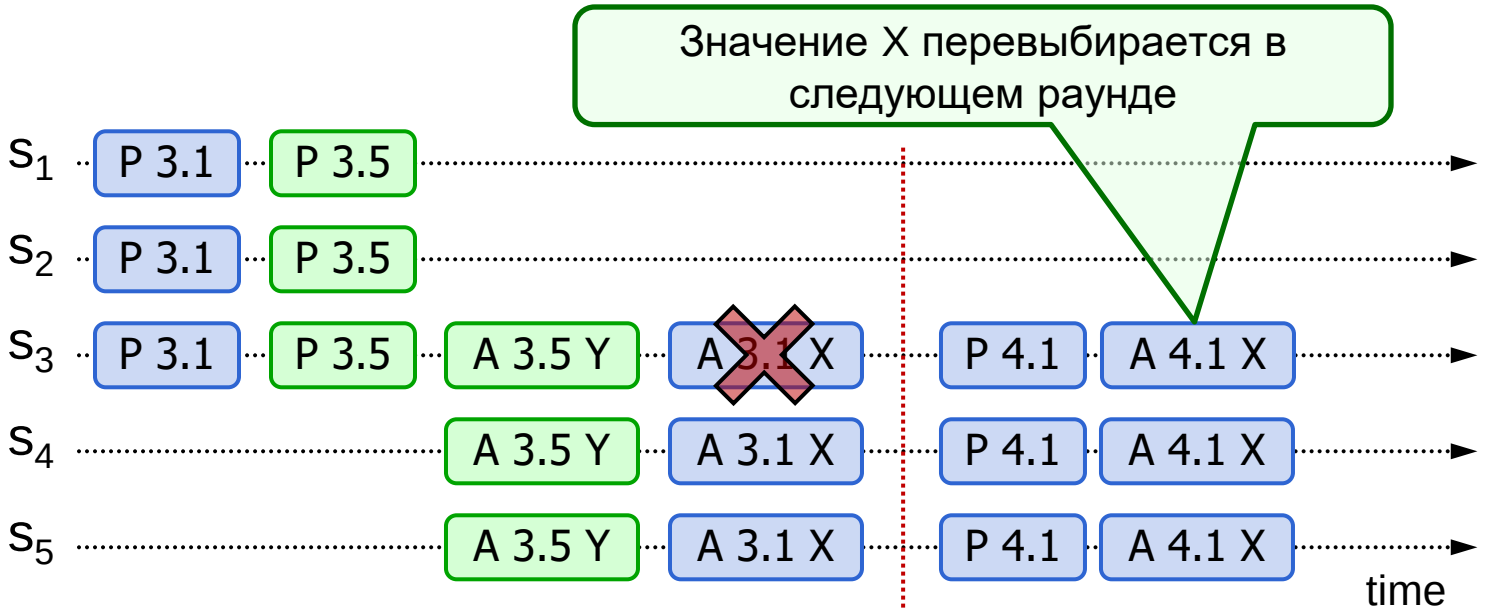


LET IT TRADE

УНИВЕРСИТЕТ ИТМО

Basic Paxos
Алгоритм acceptor-а

- Если голосование за Accept(***K***, V_k) не дает обещания не голосовать за предыдущее предложение Accept(***N***, V_N), где ***N*** < ***K***, то выбранное значение V_K может перестать быть выбранным и может пропасть



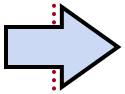
LET IT TRADE

ITIVITI
УНИВЕРСИТЕТ ИТМО

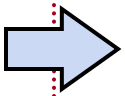
Basic Paxos

Proposers

1) Choose new proposal number n
2) Broadcast Prepare(n) to all servers



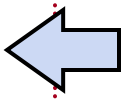
4) When responses received from majority:
   - If any acceptedValues returned, replace value with acceptedValue for highest acceptedProposal
5) Broadcast Accept(n, value) to all servers



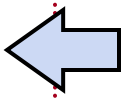
6) When responses received from majority:
   - Any rejections (result > n)? goto (1)
   - Otherwise, **value is chosen**

Acceptors

3) Respond to Prepare(n):
   - If n > minProposal then minProposal = n
   - Return(acceptedProposal, acceptedValue)



6) Respond to Accept(n, value):
   - If n ≥ minProposal then
     acceptedProposal = minProposal = n
     acceptedValue = value
   - Return(minProposal)

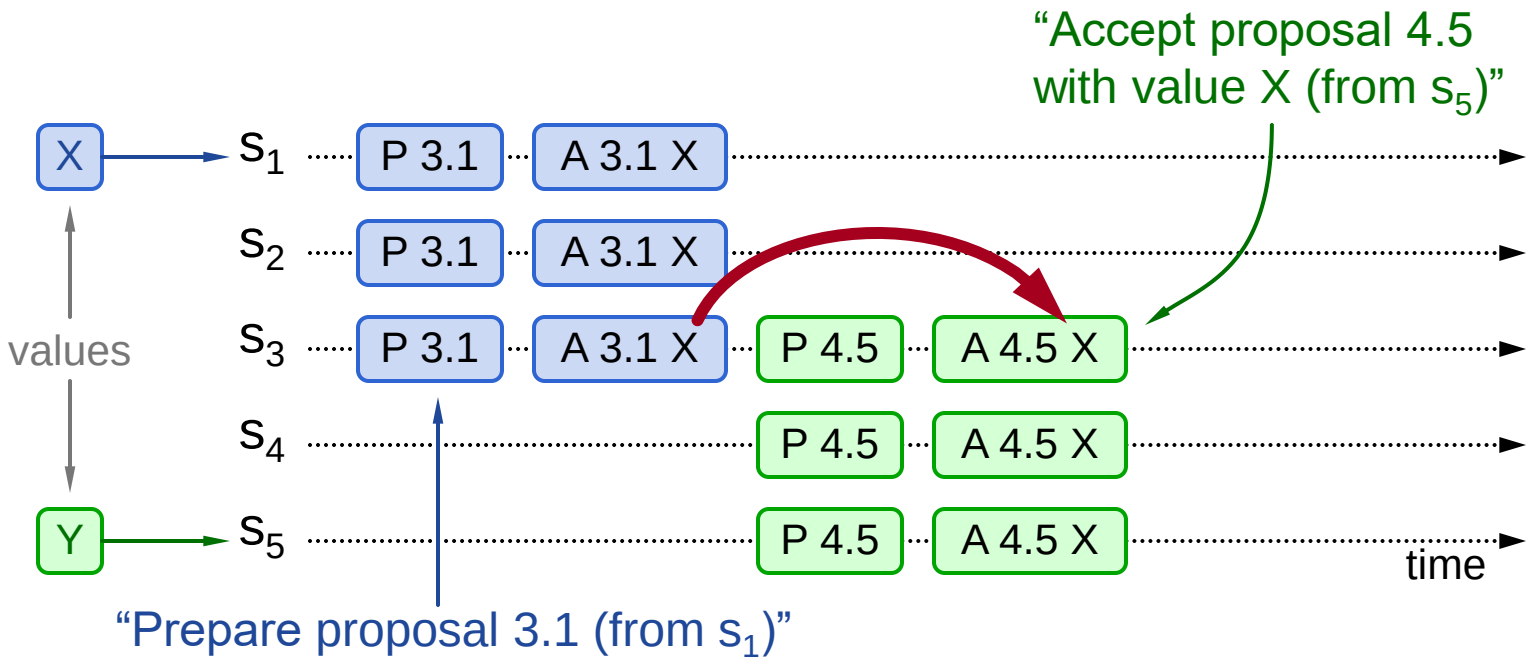


Acceptors must record minProposal, acceptedProposal, and acceptedValue on stable storage (disk)

Basic Paxos Examples

Варианты Фазы 1 Prepare следующего раунда:

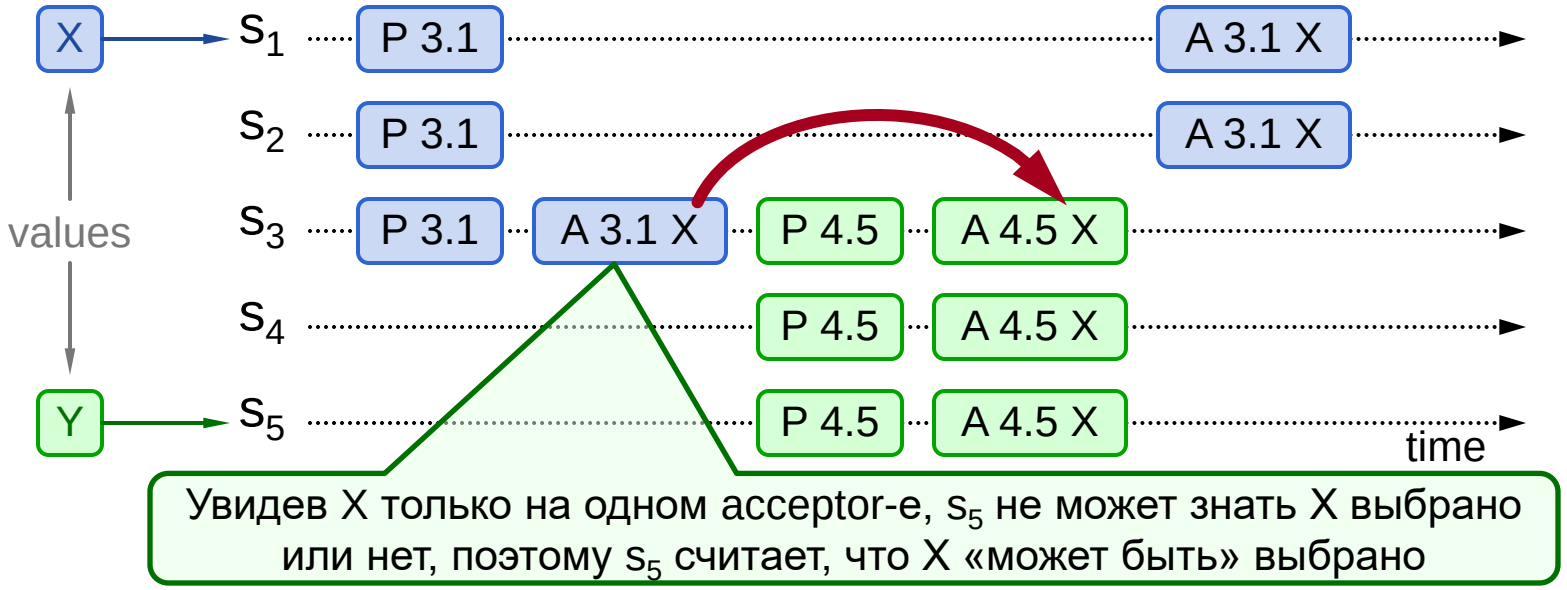
1. **Предыдущее значение уже выбрано:**
   - Следующее предложение увидит его и будет использовать
   - **Оба** предложения завершились успехом



Basic Paxos Examples, cont'd

Варианты Фазы 1 Prepare следующего раунда:

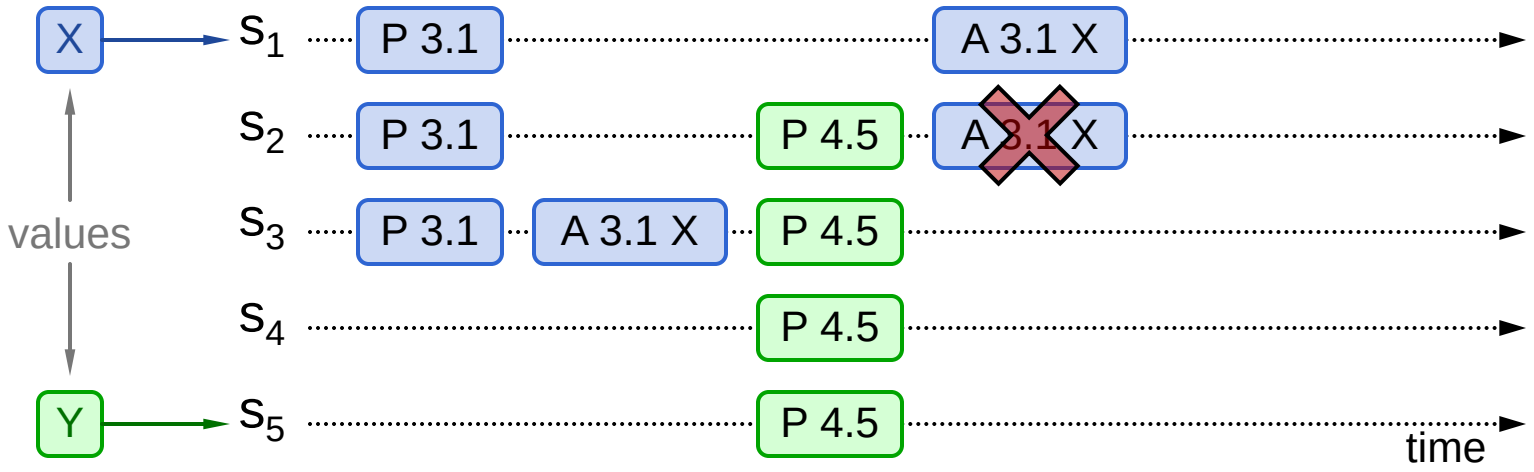
2. **Предыдущее значение еще не выбрано, но proposer увидел его на Фазе 1:**
   - Следующее предложение использует увиденное значение
   - **Оба** предложения завершились успехом



Basic Paxos Examples, cont'd

Варианты Фазы 1 Prepare следующего раунда:

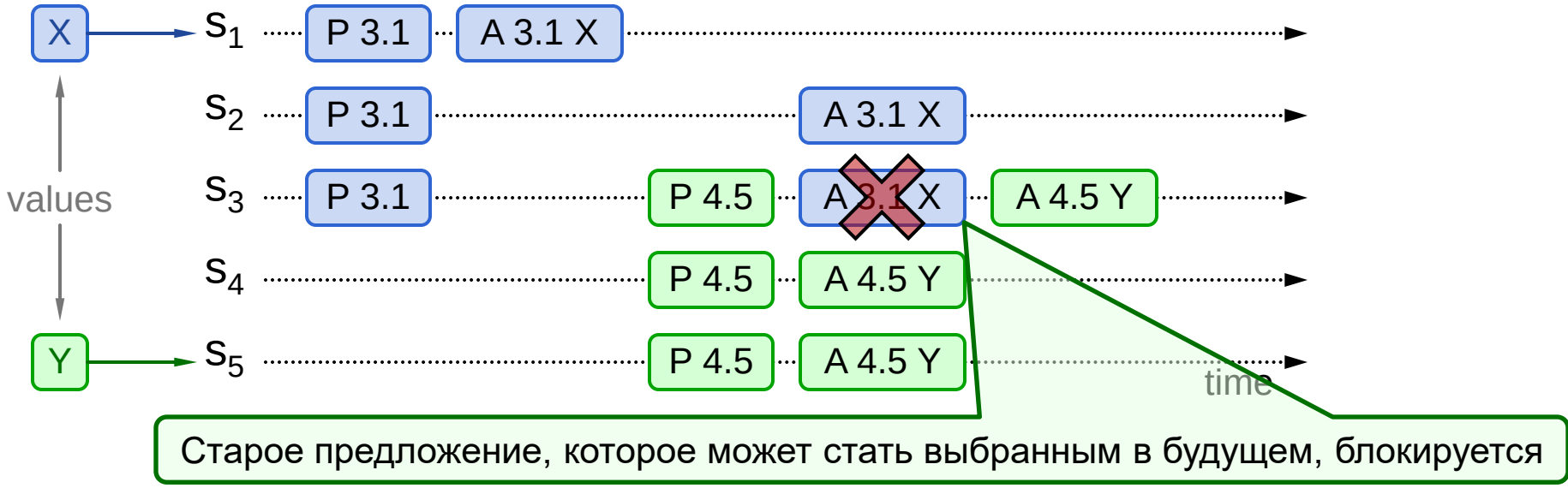
2. **Зачем новому proposer-у рассылать accept со старым значением? Почему не позволить старому proposer-у завершить свой выбор старого значения?**
   - Старый proposer может fail или может быть заблокирован новым proposer-ом и его выбор так и не состоится
   - Успехом завершается **последний** proposer



Basic Paxos Examples, cont'd

Варианты Фазы 1 Prepare следующего раунда:

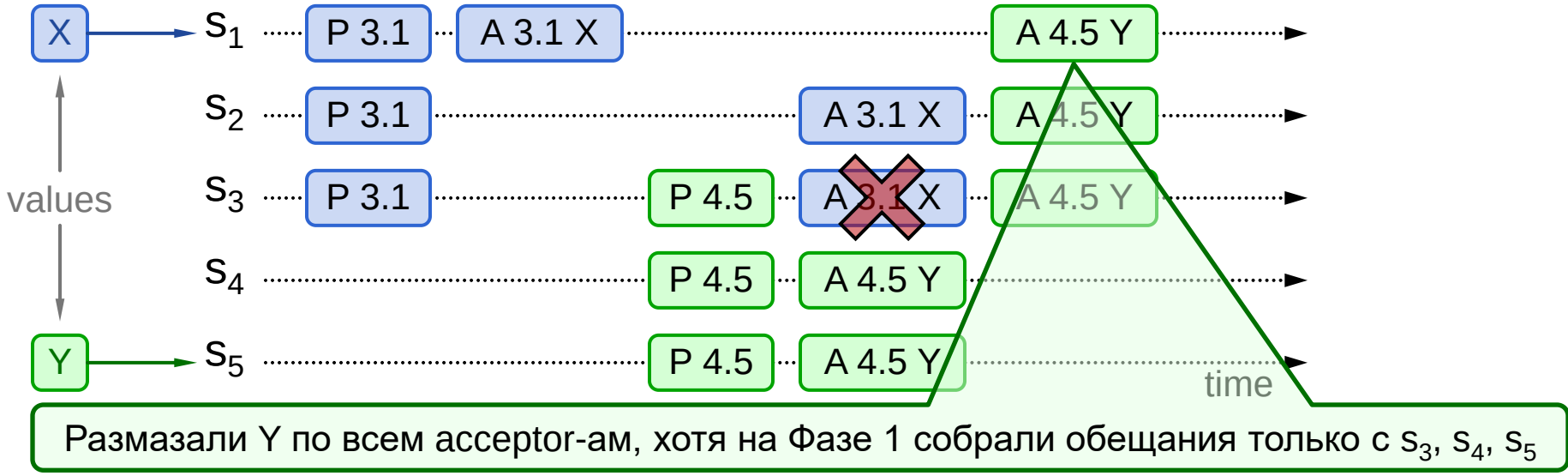
3. **Предыдущее значение еще не выбрано, и proposer не увидел его на Фазе 1:**
   - Следующий предложение будет использовать своё значение
   - Старое предложение заблокировано
   - Успехом завершается **последний** proposer



Basic Paxos Examples, cont'd

Варианты Фазы 1 Prepare следующего раунда:

3. **Предыдущее значение еще не выбрано, и proposer не увидел его на Фазе 1:**
   - Что делать s_1 и s_2 со значением Y, если они уже приняли X?
   - Ответ: проголосовать за значение Y, т.к. номер его предложения больше



Basic Paxos
Learning a chosen value

ITIVITI

- Chosen value revisited: как глядя на acceptor-ов понять, что выбор сделан?
  - Вариант 1: ***V*** accepted by majority
  - Вариант 2: (***N***, ***V***) accepted by majority

On some subtleties of Paxos

https://www.the-paper-trail.org/post/2012-11-03-on-some-subtleties-of-paxos/

«This short post tries to clear up some confusion on a part of the protocol that is poorly explained in pretty much every major description…

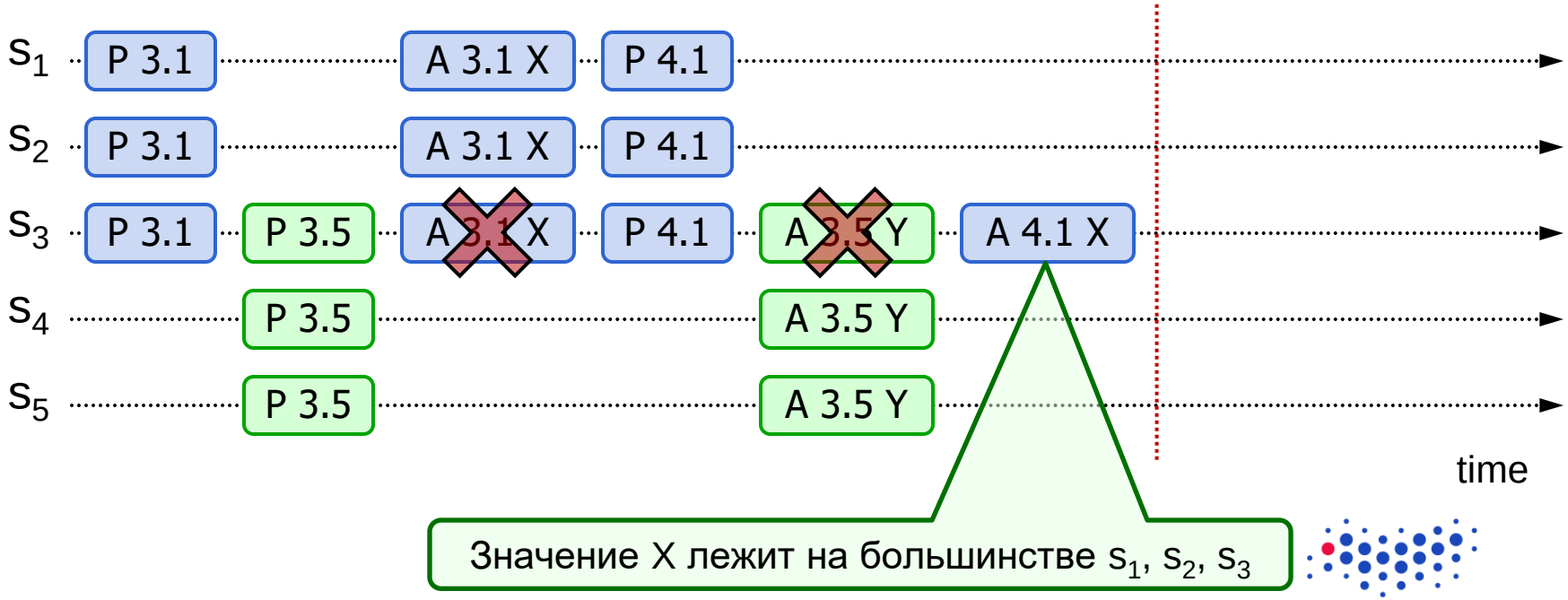
… A more interesting case is the following: suppose that a majority of acceptors have accepted a proposal with the same value, but with different sequence numbers (i.e. proposed by a different proposer). Can a learner commit to that value once it has learnt about all the acceptances? In the following section we show that it can…»

LET IT TRADE

УНИВЕРСИТЕТ ИТМО

Basic Paxos
Learning a chosen value

- Chosen value revisited: как глядя на acceptor-ов понять, что выбор сделан?
  - Вариант 1: ***V*** accepted by majority
  - Вариант 2: (***N***, ***V***) accepted by majority



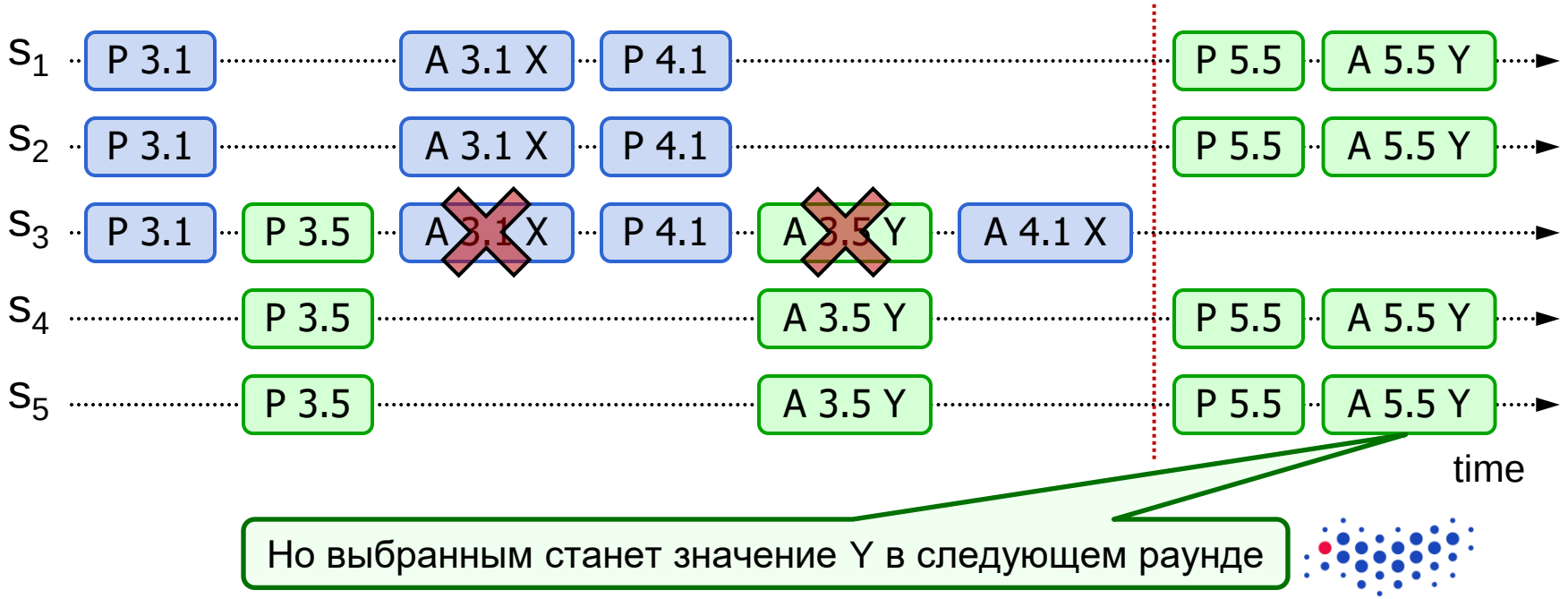
LET IT TRADE

ITIVITI
УНИВЕРСИТЕТ ИТМО

Basic Paxos
Learning a chosen value

ITIVITI

- Chosen value revisited: как глядя на acceptor-ов понять, что выбор сделан?
  - Вариант 1: ***V*** accepted by majority
  - Вариант 2: (***N***, ***V***) accepted by majority



УНИВЕРСИТЕТ ИТМО

LET IT TRADE

Basic Paxos
Доказательство корректности



- **П2б:** Если предложение ***M*** со значением V_M выбрано, то **любое сделанное** предложение с бОльшим номером ***N*** > ***M*** также должно иметь значение V_M
  - «В реальном времени» значение в меньшем раунде ***M*** может быть выбрано «после» выбора значения в большем раунде ***N***
- Пусть ***C*** – произвольное большинство acceptor-ов, которые проголосовали или проголосуют за Accept(***M***, V_M)
- Мы делаем Prepare(***N***) на другое большинство acceptor-ов ***S***, и посмотрим на acceptor ***A*** из пересечения ***C*** и ***S***
  - ***A*** точно получил Accept(***M***, V_M) *до* Prepare(***N***), т.к. иначе он проигнорировал бы Accept(***M***, V_M) и не проголосовал за него
  - По индукции по proposal number все предложения от ***M*** до ***N***-1 имеют значение V_M => на ***A*** лежит (***N''***, V_M), где ***N''*** находится в пределах от ***M*** до ***N***-1 и Prepare(***N***) вернет V_M

LET IT TRADE



УНИВЕРСИТЕТ ИТМО

Basic Paxos: Сбой acceptor-ов

ITIVITI

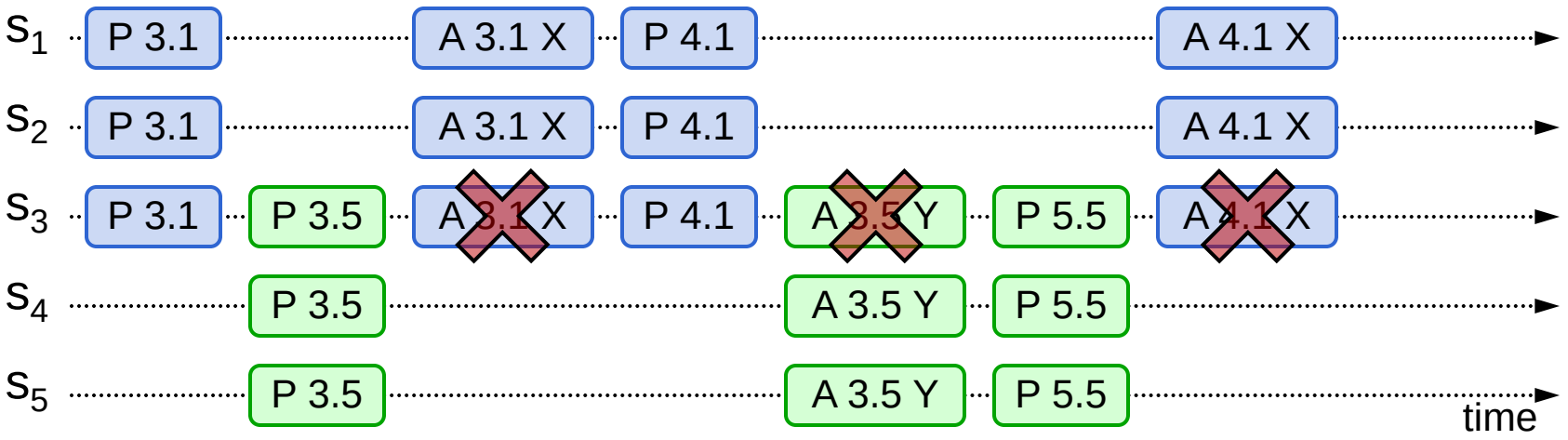
- Интересная ситуация revisited: половина acceptor-ов (s_1 и s_2) проголосовала за **red**, половина (s_4 и s_5) за **blue**
- s_3 не дышит, не успев сообщить результаты своего accept-а, т.е. алгоритм не завершился с т.з. proposer-ов **red** и **blue**
- Алгоритм оказывается блокированным на s_3?
- Пусть **red** proposal number < **blue** proposal number, тогда по **П2б** s_3 **не мог проголосовать** за **red**, т.е. **red** не выбрано
  - Значит либо s_3 не голосовал, либо проголосовал за **blue**
  - Если s_3 умер насовсем, то мы выберем **blue** в следующих раундах (размажем по остальным acceptor-ам)
  - Если s_3 оживет и на нём будет **blue**, то мы выберем **blue**
  - Если s_3 оживет и окажется непроголосовавшим, то у **red** есть шанс в следующих раундах

LET IT TRADE

УНИВЕРСИТЕТ ИТМО

Liveness

- **Competing proposers can livelock:**



- **One solution: randomized delay before restarting**
  - Give other proposers a chance to finish choosing
- **Multi-Paxos will use leader election instead**

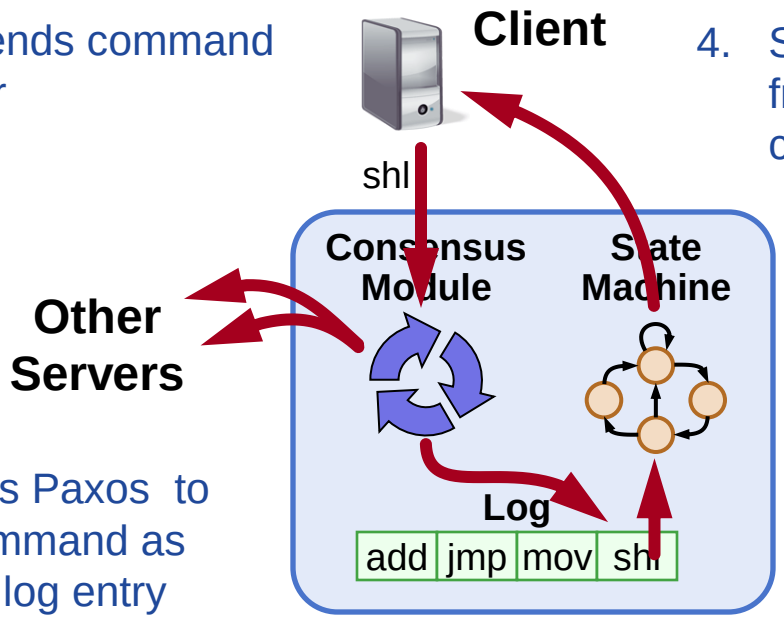
Other Notes

- Only proposer knows which value has been chosen
- If other servers want to know, must execute Paxos with their own proposal
  - Fault tolerant к message loss (pull модель)
  - Есть возможность перезапустить раунд алгоритма по таймаута, если из-за message loss не получили ответа от majority of acceptors
- Или proposer должен разослать всем chosen value сам, как только узнает факт его выбора
  - Not fault tolerant к message loss (push модель)
- Или каждый acceptor при каждом своём accept-е сообщает об этом learner-у. Когда learner узнает, что какое-то предложение выбрано большинством acceptor-ов, значит значение chosen
  - Not fault tolerant к message loss (push модель)

Multi-Paxos

- **Separate instance of Basic Paxos for each entry in the log:**
  - Add index argument to Prepare and Accept (selects entry in log)

1. Client sends command to server



4. Server returns result from state machine to client

2. Server uses Paxos to choose command as value for a log entry

3. Server waits for previous log entries to be applied, then applies new command to state machine

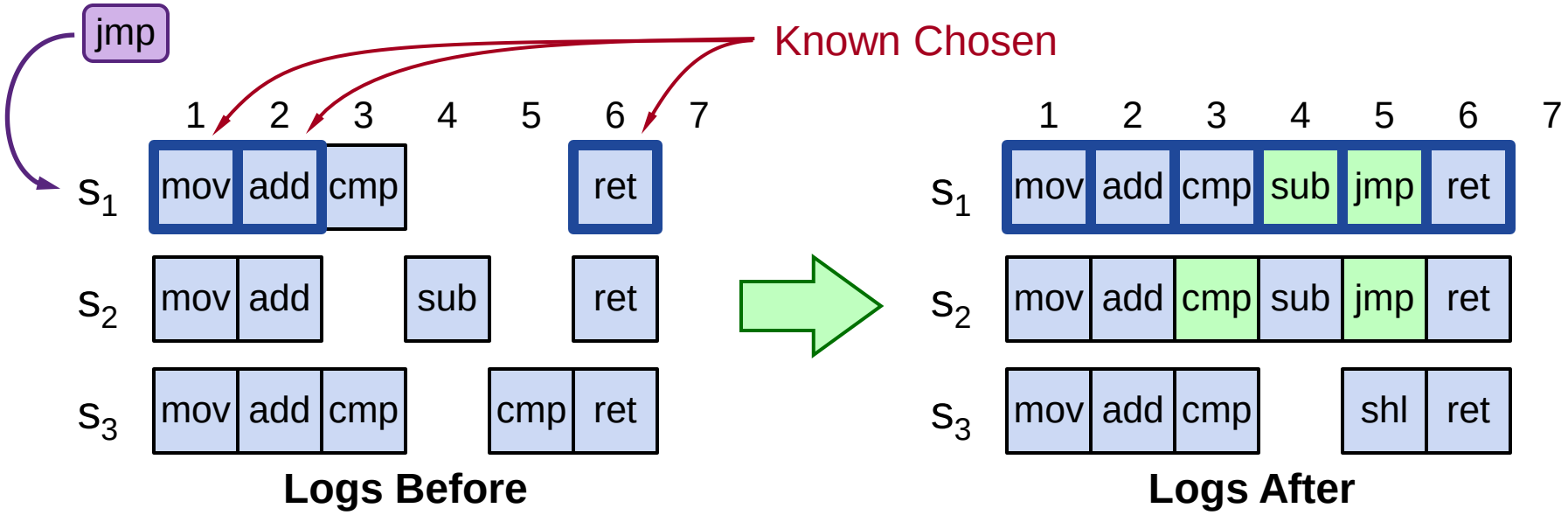
Multi-Paxos Issues

- **Which log entry to use for a given client request?**
- **Performance optimizations:**
  - Use leader to reduce proposer conflicts
  - Eliminate most Prepare requests
- **Ensuring full replication**
- **Client protocol**
- **Configuration changes**

Note: Multi-Paxos not specified precisely in literature

Selecting Log Entries

- **When request arrives from client:**
  - Find first log entry not known to be chosen
  - Run Basic Paxos to propose client's command for this index
  - Prepare returns acceptedValue?
    - Yes: finish choosing acceptedValue, start again
    - No: choose client's command



Selecting Log Entries, cont'd

- **Servers can handle multiple client requests concurrently:**
  - Select different log entries for each
- **Must apply commands to state machine in log order**

Improving Efficiency

- **Using Basic Paxos is inefficient:**
  - With multiple concurrent proposers, conflicts and restarts are likely (higher load → more conflicts)
  - 2 rounds of RPCs for each value chosen (Prepare, Accept)

Solution:

1. **Pick a leader**
   - At any given time, only one server acts as Proposer
2. **Eliminate most Prepare RPCs**
   - Prepare once for the entire log (not once per entry)
   - Most log entries can be chosen in a single round of RPCs

Leader Election

One simple approach from Lamport:

- **Let the server with highest ID act as leader**
- **Each server sends a heartbeat message to every other server every T ms**
- **If a server hasn't received heartbeat from server with higher ID in last 2T ms, it acts as leader:**
  - Accepts requests from clients
  - Acts as proposer and acceptor
- **If server not leader:**
  - Rejects client requests (redirect to leader)
  - Acts only as acceptor

Eliminating Prepares

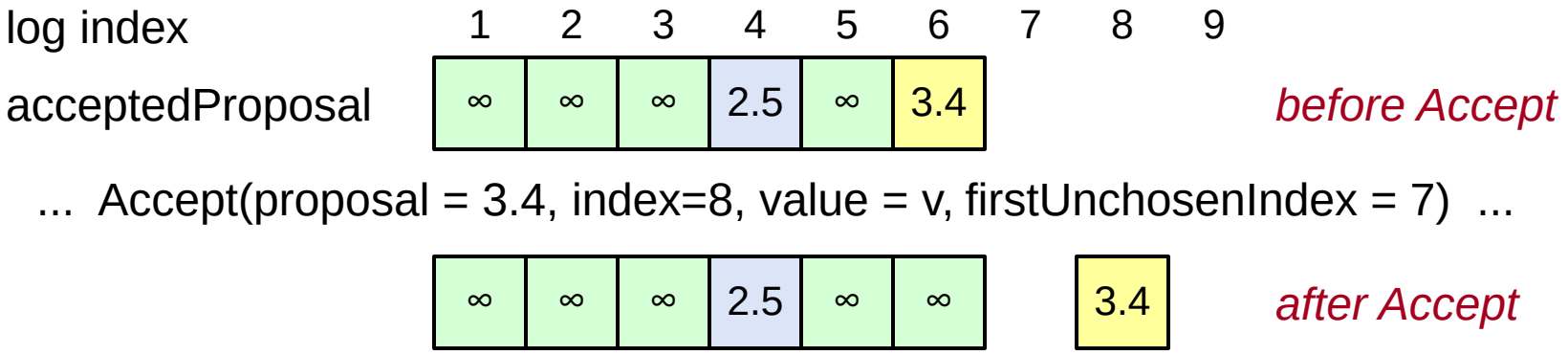
- **Why is Prepare needed?**
  - Block old proposals
    - Make proposal numbers refer to the entire log, not just one entry
  - Find out about (possibly) chosen values
    - Return highest proposal accepted for current entry
    - Also return noMoreAccepted: no proposals accepted for any log entry beyond current one
- **If acceptor responds to Prepare with noMoreAccepted, skip future Prepares with that acceptor (until Accept rejected)**
- **Once leader receives noMoreAccepted from majority of acceptors, no need for Prepare RPCs**
  - Only 1 round of RPCs needed per log entry (Accepts)

Full Disclosure

- **So far, information flow is incomplete:**
  - Log entries not fully replicated (majority only)
    Goal: full replication
  - Only proposer knows when entry is chosen
    Goal: all servers know about chosen entries
- **Solution part 1/4: keep retrying Accept RPCs until all acceptors respond (in background)**
  - Fully replicates most entries
- **Solution part 2/4: track chosen entries**
  - Mark entries that are known to be chosen:
    acceptedProposal[i] = ∞
  - Each server maintains firstUnchosenIndex: index of earliest log entry not marked as chosen

Full Disclosure, cont'd

- **Solution part 3/4: proposer tells acceptors about chosen entries**
  - Proposer includes its firstUnchosenIndex in Accept RPCs.
  - Acceptor marks all entries i chosen if:
    - i < request.firstUnchosenIndex
    - acceptedProposal[i] == request.proposal
  - Result: acceptors know about *most* chosen entries



Still don't have complete information

Full Disclosure, cont'd

- **Solution part 4/4: entries from old leaders**
  - Acceptor returns its firstUnchosenIndex in Accept replies
  - If proposer's firstUnchosenIndex > firstUnchosenIndex from response, then proposer sends Success RPC (in background)
- **Success(index, v): notifies acceptor of chosen entry:**
  - acceptedValue[index] = v
  - acceptedProposal[index] = ∞
  - return firstUnchosenIndex
  - Proposer sends additional Success RPCs, if needed

Client Protocol

- **Send commands to leader**
  - If leader unknown, contact any server
  - If contacted server not leader, it will redirect to leader
- **Leader does not respond until command has been chosen for log entry and executed by leader's state machine**
- **If request times out (e.g., leader crash):**
  - Client reissues command to some other server
  - Eventually redirected to new leader
  - Retry request with new leader

Client Protocol, cont'd

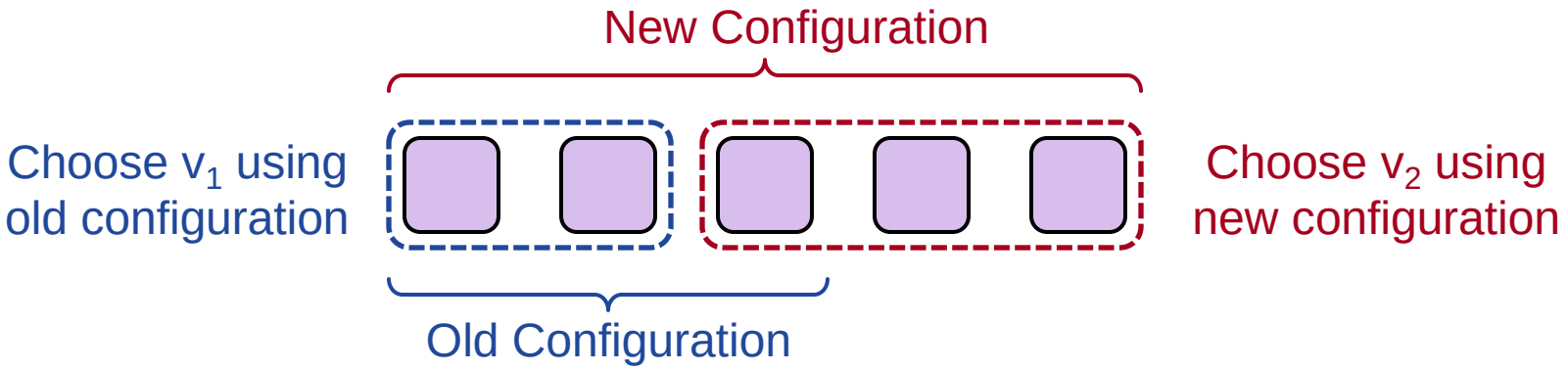
- **What if leader crashes after executing command but before responding?**
  - Must not execute command twice
- **Solution: client embeds a unique id in each command**
  - Server includes id in log entry
  - State machine records most recent command executed for each client
  - Before executing command, state machine checks to see if command already executed, if so:
    - Ignore new command
    - Return response from old command
- **Result: exactly-once semantics as long as client doesn't crash**

Configuration Changes

- **System configuration:**
  - ID, address for each server
  - Determines what constitutes a majority
- **Consensus mechanism must support changes in the configuration:**
  - Replace failed machine
  - Change degree of replication

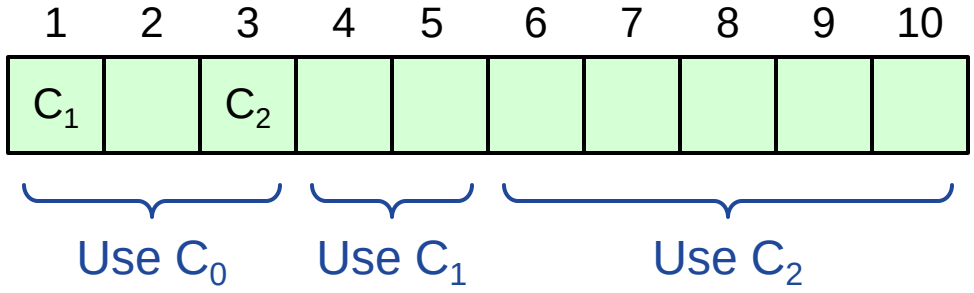
Configuration Changes, cont'd

- **Safety requirement:**
  - During configuration changes, it must not be possible for different majorities to choose different values for the same log entry:



Configuration Changes, cont'd

- **Paxos solution: use the log to manage configuration changes:**
  - Configuration is stored as a log entry
  - Replicated just like any other log entry
  - Configuration for choosing entry i determined by entry i-α. Suppose α = 3:



- **Notes:**
  - α limits concurrency: can't choose entry i+α until entry i chosen
  - Issue no-op commands if needed to complete change quickly

Paxos Summary

- **Paxos про конкуренцию лидеров, а не про выбор значения**
- **Basic Paxos:**
  - Prepare phase
  - Accept phase
- **Multi-Paxos:**
  - Choosing log entries
  - Leader election
  - Eliminating most Prepare requests
  - Full information propagation
- **Client protocol**
- **Configuration changes**